Practical Worksheet: Testing & Debugging an Android App

Unit testing · UI testing · Emulators vs real devices · Debugging tools

Before you start — setup checklist

Tick each item. Ask the demonstrator if any step fails; do not start Part A until all four are ticked.

☐	Android Studio opens, and a project (new or existing) builds successfully.
☐	An AVD exists — API level 34, 6.1-inch screen, 2 GB RAM — and boots to the home screen.
☐	The src/test and src/androidTest source folders are both present in the Project view.
☐	The test dependencies below are in app/build.gradle and Gradle has synced without errors.

```
dependencies {
    testImplementation 'junit:junit:4.13.2'
    testImplementation 'org.mockito:mockito-core:5.11.0'

    androidTestImplementation 'androidx.test.ext:junit:1.1.5'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'
}
```

Remember the split: `src/test` holds local tests that run on the JVM; `src/androidTest` holds instrumented tests that need an emulator or a real device. Putting a test in the wrong folder is the most common reason it refuses to run.

Part A — Unit testing with JUnit

A unit test checks that a single method produces the expected output for a given input, with no emulator involved. Everything in this part runs on the JVM in milliseconds.

A1 Create the class under test *(15 min)*

In your main source set, create a Calculator class with add, subtract, multiply and divide methods that take two integers.

```
class Calculator {
    fun add(a: Int, b: Int): Int = a + b
    fun subtract(a: Int, b: Int): Int = a - b
    fun multiply(a: Int, b: Int): Int = a * b
    fun divide(a: Int, b: Int): Int = a / b
}
```

In `src/test`, create `CalculatorTest` and copy in the test below. Run it and confirm it passes.

```
class CalculatorTest {

    @Test
    fun addition_isCorrect() {
        val calculator = Calculator()          // Arrange
        val result = calculator.add(2, 3)      // Act
        assertEquals(5, result)                // Assert
    }
}
```

Q A1. The three comments above mark the Arrange-Act-Assert pattern. In your own words, what does each of the three steps do?

A2 Write your own tests and assertions *(25 min)*

Add four more @Test methods to CalculatorTest. Give each one a descriptive name in the style methodName_condition_expectedResult.

#	What to test	Assertion to use	Pass / Fail
1	subtract(10, 4) returns 6	assertEquals	
2	multiply(0, 99) returns 0	assertEquals	
3	add(-2, -3) returns a negative number	assertTrue	
4	divide(10, 0) throws an exception	assertThrows	

Test 4 is the interesting one. Complete it:

```
@Test
fun divide_byZero_throwsException() {
    val calculator = Calculator()

    assertThrows(ArithmeticException::class.java) {
        // TODO: call the method that should throw
    }
}
```

Q A2. Which exception class is actually thrown, and why is asserting on the exception better than wrapping the call in a try/catch that does nothing?

A3 Break something on purpose *(10 min)*

Change `add` to return `a + b + 1` and re-run the suite.

How many tests failed?	
Copy the failure message the test report showed you	

Now restore the correct implementation and confirm the suite is green again. This is exactly what a regression looks like in a real project — the test caught it in seconds, without anyone opening the app.

Part B — Isolating dependencies with Mockito

A ViewModel that calls a repository cannot be unit tested honestly if the repository hits a real network or database. A mock replaces that dependency with controlled, predictable behaviour.

B1 Mock a repository *(30 min)*

Create the two classes below, then write the test.

```
data class User(val id: Int, val name: String)

interface UserRepository {
    fun getUser(id: Int): User
}

class UserViewModel(private val repository: UserRepository) {
    var userName: String? = null; private set

    fun loadUser(id: Int) {
        userName = repository.getUser(id).name
    }
}
```

Now write the test in `src/test`:

```
@Test
fun loadUser_setsUserName() {
    val mockRepository = mock(UserRepository::class.java)
    `when`(mockRepository.getUser(1))
        .thenReturn(User(1, "Nimal"))

    val viewModel = UserViewModel(mockRepository)
    viewModel.loadUser(1)

    verify(mockRepository).getUser(1)
    assertEquals("Nimal", viewModel.userName)
}
```

Q B1a	What does the `` `when`(...).thenReturn(...) `` line do?
Q B1b	`verify` and `assertEquals` check two different things here. What does each one check?
Q B1c	This test never touches a network. Give one reason that is an advantage, and one thing it therefore cannot tell you.

Part C — UI testing with Espresso

UI tests verify what the user actually sees and touches. They are instrumented tests, so they need an emulator or a real device. Every Espresso statement follows the same three moves: find a view, act on it, check the result.

C1 Build a login screen *(20 min)*

Create a `LoginActivity` with three views: an email field (`R.id.emailInput`), a password field (`R.id.passwordInput`) and a button (`R.id.loginButton`). Show the message `"Invalid email"` in a `R.id.errorText` view when the email does not contain an @ symbol, or when either field is empty.

C2 Write and run the UI test *(30 min)*

In `src/androidTest`, create the class below and complete the missing assertion.

```
@RunWith(AndroidJUnit4::class)
class LoginActivityTest {

    @get:Rule
    val activityRule = ActivityScenarioRule(LoginActivity::class.java)

    @Test
    fun login_withInvalidEmail_showsError() {
        onView(withId(R.id.emailInput))
            .perform(typeText("not-an-email"), closeSoftKeyboard())
        onView(withId(R.id.passwordInput))
            .perform(typeText("pass123"), closeSoftKeyboard())
        onView(withId(R.id.loginButton)).perform(click())

        // TODO: check that R.id.errorText is displayed
    }
}
```

Then write a second test of your own: entering a valid email and password should hide the error message.

Espresso call	Find / act / check?	What it does in this test
`onView(withId(...))`		
`perform(click())`		
`check(matches(...))`		

Q C2. Time one run of this UI test and one run of your CalculatorTest from Part A. Record both, then explain the difference.

	Time taken	Where it ran (JVM / emulator / device)
CalculatorTest (unit)		
LoginActivityTest (UI)		

Why is one so much slower than the other?

Part D — Emulators vs real devices

Emulators are fast to create and cost nothing, but they cannot perfectly reproduce a manufacturer's hardware or OS customisations. This part is about seeing that difference for yourself.

D1 Run the same test on two targets *(20 min)*

Run your Espresso test on the API 34 emulator, then on a real Android device connected over USB with USB debugging enabled. If no physical device is available, use a second AVD with a different API level and screen size, and note that substitution in the table.

	Emulator	Real device (or 2nd AVD)
Model / AVD name		
API level		
Screen size		
Test result		
Time to run		
Anything that behaved differently		

Q D1. Your app runs smoothly on the emulator but lags badly on a low-end phone. Explain why the emulator did not reveal this, and state at what point in a project a real device becomes essential.

Part E — Debugging tools

Each tool answers a different question. Logcat tells you what the app reported, the debugger tells you what the variables actually hold, the Layout Inspector tells you what the view hierarchy really looks like, and the Profiler tells you what the app is costing.

E1 Instrument the login flow with Logcat *(15 min)*

Add log statements to your login method at three different levels, then filter Logcat by the tag.

```
class LoginActivity : AppCompatActivity() {

    private fun login(email: String, password: String) {
        Log.d("LoginActivity", "Attempting login for $email")

        if (isValid(email, password)) {
            Log.i("LoginActivity", "Login succeeded")
        } else {
            Log.e("LoginActivity", "Login failed: invalid input")
        }
    }
}
```

Log level	Letter in Logcat	When you would use it
Verbose		
Debug		
Info		
Warn		
Error		

Q E1. Why should a log message never contain a user's password, even at Debug level?

E2 Find a planted bug with the debugger *(30 min)*

Replace your validation method with the version below. It contains a deliberate defect: the error message appears even for valid input.

```
private fun isValid(email: String, password: String): Boolean {
    if (email.isEmpty() || password.isEmpty()) return false
    if (email.contains("@") && password.length > 6) return false   // bug
    return true
}
```

Do not fix it by reading the code. Work through the debugging loop and record what you find at each step.

#	Step	What you observed

1	Reproduce the bug — what input triggers it?	
2	Set a breakpoint on the first line of isValid	
3	Run in debug mode and inspect email and password	
4	Step over each line — which line returns the wrong value?	
5	State your hypothesis, then apply the fix	
6	Re-run your Espresso test to confirm	

Q E2a. Set a conditional breakpoint that pauses only when the email field is empty. Write the condition you used:

Q E2b. Explain the difference between Step Over and Step Into.

E3 Layout Inspector and Profiler *(20 min)*

With the app running, open each tool and record one concrete observation from each.

Tool	Question it answers	What you found in your app
Layout Inspector		
Profiler — CPU		
Profiler — Memory		

Q E3. A screen feels slow to scroll. Name the two tools you would reach for first, and say what each would tell you.

Part F — Review questions

Answer from what you did today, not from the lecture slides.

01 What is the difference between a unit test and a UI test, in terms of where each one runs?

02 Why do unit tests run faster than UI tests?

03 What role does Mockito play in a unit test, and what problem does it solve?

04 You add a test that needs a real Activity. Which source folder does it belong in, and why?

05 Give one reason to test on a real device instead of only an emulator.

06 A test passes on your machine but fails on a classmate's. Suggest two possible causes.

Submission checklist

☐	CalculatorTest with at least five passing test methods.
☐	UserViewModel test using a Mockito mock, with both verify and assertEquals.
☐	Two Espresso tests for LoginActivity, both passing on an emulator.
☐	Screenshot of the test report showing all tests green.
☐	Screenshot of Logcat filtered by your tag, showing all three log levels.
☐	This worksheet completed, including the Part D comparison table.